

Relazione - Versione 5: Memory Manager

Introduzione

La Versione 5 introduce il Memory Manager.

Nella versione precedente, il progetto ha introdotto il Planner. L'agente era già in grado di selezionare uno strumento, creare un piano di esecuzione, estrarre gli argomenti richiesti, eseguire lo strumento selezionato e memorizzare l'interazione.

La Versione 5 mantiene questa struttura, ma aggiunge un nuovo modo di gestire la memoria: ora l'Agent non scrive più direttamente nella lista memory.

Questa versione è ancora basata su regole. Non viene utilizzato alcun LLM, non viene introdotto alcun database e non viene introdotto alcun framework esterno.

Obiettivo della Versione 5

L'obiettivo principale della Versione 5 è introdurre una gestione più pulita della memoria.

L'Agent non deve più occuparsi direttamente del salvataggio, della lettura o della pulizia della memoria. Deve delegare queste operazioni a un componente dedicato.

Il Memory Manager introduce funzioni semplici per gestire la memoria:

- `save_to_memory(state);`
- `get_memory();`
- `get_last_interaction();`
- `clear_memory();`

L'idea importante è semplice: la memoria non deve solo esistere, deve essere gestita.

Cosa cambia rispetto alla Versione 4

La Versione 4 si concentrava sul Planner. Il Planner ha introdotto un piano prima del parsing e dell'esecuzione.

La Versione 5 non modifica il Planner come concetto principale. Il Planner continua a creare un piano con lo strumento selezionato, il motivo del suo utilizzo e gli argomenti richiesti.

Il nuovo componente è il Memory Manager.

Il flusso cambia da:

Richiesta → Router → Planner → Generic Parser → Executor → Memoria

a:

Richiesta → Router → Planner → Generic Parser → Executor → Memory Manager → Memoria

L'Executor continua a produrre il risultato. Il Memory Manager si occupa quindi di salvare, leggere o pulire lo stato dell'interazione.

Nuovo concetto: Memory Manager

Il Memory Manager è un nuovo componente che gestisce le operazioni sulla memoria.

In questa versione, la memoria rimane una semplice lista. La differenza è che non viene più usata direttamente dall'Agent.

Se uno stato deve essere salvato, l'Agent non chiama più direttamente:

```
memory.append(state)
```

ma delega al Memory Manager:

```
save_to_memory(state)
```

Questo rende il ruolo dell'Agent più pulito. L'Agent coordina il flusso, mentre il Memory Manager gestisce la memoria.

Funzioni del Memory Manager

La Versione 5 introduce funzioni semplici e leggibili.

La funzione `save_to_memory(state)` salva lo stato corrente nella memoria.

La funzione `get_memory()` restituisce tutta la memoria.

La funzione `get_last_interaction()` restituisce l'ultima interazione salvata. Se la memoria è vuota, restituisce `None`.

La funzione `clear_memory()` pulisce la memoria.

Queste funzioni rendono la memoria più facile da controllare durante i test e preparano il progetto a gestioni più avanzate nelle versioni future.

Architettura completa nella Versione 5

Il flusso completo della Versione 5 è:

```
Richiesta → Router → Planner → Generic Parser → Executor → Memory Manager → Memoria
↓
Output
```

Le responsabilità principali ora sono più chiare:

Router = seleziona lo strumento

Planner = crea il piano di esecuzione

Parser = estrae gli argomenti

Executor = convalida ed esegue

Memory Manager = salva, legge e pulisce la memoria

Agent = coordina

Memory = conserva lo stato

L'Agent è ancora il coordinatore. Ora coordina pianificazione, parsing, esecuzione e memorizzazione attraverso il Memory Manager.

Stato dell'Agent nella Versione 5

Lo stato dell'agente mantiene la stessa struttura introdotta nella Versione 4.

Uno stato tipico appare così:

```
{
  "request": "Quanto fa 2 + 3?",
  "plan": {
    "tool": "addition",
    "reason": "Somma due numeri.",
    "needs_arguments": ["a", "b"]
  },
  "action": "addition",
  "arguments": {"a": 2, "b": 3},
  "result": 5
}
```

La differenza è che questo stato viene salvato tramite il Memory Manager, non con una scrittura diretta nella lista memory.

Richieste non supportate

Anche le richieste non supportate vengono gestite correttamente.

Per esempio, la richiesta:

```
agent("Che tempo fa a Roma?")
```

non corrisponde ad alcuno strumento disponibile.

Il Planner restituisce un piano senza strumento selezionato e l'Agent restituisce un risultato alternativo sicuro.

La novità della Versione 5 è che anche questa interazione viene salvata tramite il Memory Manager.

Questo è importante perché la memoria deve registrare sia le richieste supportate sia quelle non supportate.

Test

I test verificano che l'agente continui a funzionare dopo l'aggiunta del Memory Manager.

I principali casi di test sono:

```
clear_memory()
agent("Quanto fa 2 + 3?")
agent("Ciao, quanto fa 2 + 3?")
agent("Ciao, mi chiamo luca.")
agent("Che tempo fa a Roma?")
get_memory()
get_last_interaction()
```

Questi test verificano che:

- il Router continui a dare priorità all'intento principale;
 - il Planner continui a creare un piano prima dell'esecuzione;
 - il Parser continui a estrarre gli argomenti corretti;
 - l'Executor continui a convalidare ed eseguire lo strumento selezionato;
 - le richieste non supportate vengano gestite senza selezionare uno strumento;
 - il Memory Manager salvi, legga e pulisca la memoria;
 - la memoria conservi lo stato completo delle interazioni.
-

Cosa insegna la Versione 5

La Versione 5 insegna un importante principio di progettazione degli agenti:

la memoria deve essere gestita da un componente dedicato

Il Memory Manager è ancora semplice, ma introduce una separazione importante tra coordinazione e gestione dello stato.

Questo rende l'architettura più facile da estendere in futuro, soprattutto quando la memoria diventerà più complessa o quando il progetto passerà a una struttura più professionale.

Limitazioni

La Versione 5 migliora l'architettura, ma rimangono alcune limitazioni:

- il Router è ancora basato su regole;
- il Planner è ancora basato su regole;
- il Parser utilizza ancora regole di estrazione manuali;
- la memoria è ancora una semplice lista;
- il Memory Manager non usa ancora database o storage esterno;
- non esiste ancora un Tool Registry;
- non è ancora presente alcun LLM;
- l'agente esegue ancora una sola azione alla volta.

Queste limitazioni sono previste e verranno affrontate gradualmente nelle versioni successive.

Conclusioni

La Versione 5 è un'evoluzione naturale della Versione 4.

Il Planner rimane invariato, ma il nuovo elemento centrale è il Memory Manager.

L'Agent non scrive più direttamente nella memoria. Ora delega il salvataggio, la lettura e la pulizia della memoria a funzioni dedicate.

Questo rende l'agente più pulito, più ordinato e più vicino a una struttura professionale.

Il prossimo passo sarà la Versione 6, nella quale il progetto introdurrà un Tool Registry.